

UNIVERSIDADE DE FORTALEZA

M907 – Sistemas Distribuídos

Prof. Nabor das Chagas Mendonça

RELATÓRIO TÉCNICO – SISTEMAS DISTRIBUÍDOS

Etapa 2 – Testes de Desempenho

Sock Shop – Microservices Demo Application

Equipe

Izabella Maria Pinheiro Amorim – Matrícula: 2527553

Letícia Vasconcelos Machado – Matrícula: 2418950

Fortaleza, junho de 2026

1. Introdução

Esta segunda etapa do relatório técnico dá continuidade ao trabalho iniciado na Etapa 1, na qual foi realizada a implantação da aplicação Sock Shop em ambiente Kubernetes local utilizando o Docker Desktop em um MacBook Pro com processador Apple M2. Com a aplicação operacional, o objetivo desta etapa é submetê-la a testes de desempenho sob três níveis de carga controlada, permitindo observar e quantificar a degradação progressiva do desempenho conforme o número de usuários simultâneos aumenta.

Os testes foram conduzidos pelas integrantes Izabella Amorim e Letícia Vasconcelos, utilizando a ferramenta Locust para geração de carga. Foram definidos três níveis de carga — baixa, média e alta —, sendo a carga alta calibrada experimentalmente para produzir uma taxa de falha não nula, conforme orientações da atividade. O relatório documenta a metodologia adotada, os parâmetros configurados, os resultados obtidos e uma análise comparativa entre os três cenários.

2. Aplicação e Ambiente Experimental

2.1 Aplicação Escolhida

Nome: Sock Shop – Microservices Demo Application

Repositório: <https://github.com/microservices-demo/microservices-demo>

O Sock Shop é uma aplicação de demonstração de e-commerce desenvolvida pela Weaveworks, composta por múltiplos serviços independentes que se comunicam via HTTP/REST e mensageria assíncrona com RabbitMQ. Foi selecionado por sua alta adequação para uso em laboratório: suporte nativo a Kubernetes, manifesto de implantação completo, diversidade tecnológica entre os serviços e fluxo funcional representativo para geração de carga.

2.2 Ambiente Experimental

Componente	Descrição
Hardware	MacBook Pro – Apple M2 (ARM64), 16 GB RAM
Sistema Operacional	macOS Sequoia 15.x
Orquestrador Kubernetes	Docker Desktop Kubernetes (cluster local, nó único)
Versão do Kubernetes	v1.29.x
Ferramenta de carga	Locust 2.x (execução local via pip3)
Acesso à aplicação	http://localhost (front-end na porta 80)
Integrantes executoras	Izabella Amorim e Letícia Vasconcelos

Observações sobre a Implantação

Devido à arquitetura ARM64 do processador M2, algumas imagens Docker originais do Sock Shop não possuem suporte nativo a essa plataforma. Os seguintes ajustes foram realizados nos manifestos de implantação:

- Substituição das imagens de carts-db e orders-db por mongo:4.4, compatível com ARM64;
- Substituição da imagem do rabbitmq por rabbitmq:3.8-management-alpine;
- Substituição da imagem do session-db por redis:alpine.

Ao momento dos testes, todos os serviços principais estavam operacionais: front-end, catalogue, catalogue-db, carts, orders, payment, shipping, user, user-db, queue-master, carts-db, orders-db, rabbitmq e session-db.

3. Cenário Funcional e Script de Carga

3.1 Cenário Definido

O cenário funcional simula um usuário típico navegando pela loja virtual, com ações ponderadas por frequência de uso real em sistemas de e-commerce:

Ação	Endpoint	Peso	% Aproximada
Acessar página inicial	GET /	10	~47%
Navegar pelo catálogo	GET /catalogue	5	~24%
Ver produto específico	GET /catalogue/{id}	3	~14%
Acessar carrinho	GET /cart	2	~10%
Fazer login	GET /login	1	~5%

Os pesos refletem o comportamento real de usuários em e-commerce: a maioria acessa a página inicial e navega pelo catálogo, enquanto apenas uma fração realiza login ou acessa o carrinho. O intervalo de espera entre ações foi configurado entre 1 e 3 segundos, simulando o tempo de leitura e decisão do usuário.

3.2 Ferramenta Utilizada

A ferramenta de geração de carga utilizada foi o Locust, instalado localmente via pip3. O Locust foi escolhido por ser open source, baseado em Python, possuir interface web intuitiva para configuração e monitoramento em tempo real, e gerar relatórios detalhados em HTML e CSV, facilitando a coleta das métricas obrigatórias.

3.3 Script de Carga (locustfile.py)

```
from locust import HttpUser, task, between

class SockShopUser(HttpUser):
    wait_time = between(1, 3)
```

```

@task(10)
def index(self):
    self.client.get("/")

@task(5)
def catalogue(self):
    self.client.get("/catalogue")

@task(3)
def product(self):
    self.client.get("/catalogue/3395a43e-2d88-40de-b95f-e00e1502085b")

@task(2)
def cart(self):
    self.client.get("/cart")

@task(1)
def login(self):
    self.client.get("/login", auth=("user", "password"))

```

4. Definição dos Níveis de Carga

Conforme orientações da atividade, os testes de desempenho foram executados em três níveis de carga, definidos experimentalmente. A tabela a seguir apresenta os parâmetros de cada nível e a justificativa das escolhas.

Nível	Usuários	Ramp-up	Duração	Taxa de Falha Observada
Baixa	100	10 us./s	10 minutos	0,00%
Média	1.000	50 us./s	10 minutos	~0,02%
Alta	3.000	100 us./s	10 minutos	~6,20%

4.1 Carga Baixa – 100 usuários

A carga baixa foi definida com 100 usuários simultâneos, representando uma situação em que a aplicação opera sem pressão significativa. Nesse nível, a aplicação respondeu sem nenhuma falha e com latência na casa de milissegundos, confirmando que 100 usuários fica confortavelmente abaixo do limite de capacidade do ambiente local.

4.2 Carga Média – 1.000 usuários

A carga média foi estabelecida com 1.000 usuários simultâneos. Os tempos de resposta ficaram em patamar intermediário: a mediana da página inicial passou de 5 ms para 17 ms, e o p95 saltou de 13 ms para 770 ms. A taxa de falha manteve-se próxima de zero (0,02%), indicando que a aplicação ainda processava as requisições, porém com degradação visível nos percentis superiores — posicionando este nível adequadamente entre as cargas baixa e alta.

4.3 Carga Alta – 3.000 usuários

A carga alta foi determinada experimentalmente. Após verificar que 1.000 usuários produzia taxa de falha próxima de zero (o que, conforme o enunciado, indica que o limite não havia sido atingido), foi realizado um teste com 3.000 usuários. Nesse nível, a taxa de falha global atingiu aproximadamente 6,2%, situando-se dentro da faixa aceitável definida pela atividade (entre 0% e 15%). Qualquer valor acima de 5.000 usuários resultou em taxa de falha superior a 20%, ultrapassando o limite superior recomendado.

5. Resultados dos Testes de Desempenho

Integrante executora: Izabella Maria Pinheiro Amorim e Letícia Vasconcelos Machado

Aplicação: Sock Shop | **Duração de cada teste:** 10 minutos

5.1 Carga Baixa – 100 Usuários Simultâneos

Com 100 usuários, a aplicação apresentou desempenho excelente, sem falhas em nenhum endpoint e com tempos de resposta consistentemente baixos.

Endpoint	Requisições	Falhas	Média (ms)	Mediana (ms)	Máx (ms)	P95 (ms)	RPS
GET /	11.687	0	6,6	5	180	13	23,49
GET /catalogue	5.799	0	10,1	8	184	18	11,66
GET /catalogue/{id}	3.583	0	41,4	48	164	56	7,20
GET /cart	2.275	0	47,9	50	550	65	4,57
GET /login	1.172	0	56,3	57	482	83	2,36
TOTAL	24.516	0	—	—	—	—	49,28

5.2 Carga Média – 1.000 Usuários Simultâneos

Com 1.000 usuários, foram observadas degradações significativas nos percentis superiores, embora o número absoluto de falhas tenha permanecido muito baixo (12 falhas em ~73.000 requisições).

Endpoint	Requisições	Falhas	Média (ms)	Mediana (ms)	Máx (ms)	P95 (ms)	RPS
GET /	34.894	0	1.191	17	41.961	770	109,19
GET /catalogue	17.767	6	1.906	30	43.744	19.000	55,60
GET /catalogue/{id}	10.687	2	1.853	66	43.689	17.000	33,44
GET /cart	6.936	3	2.061	68	47.503	21.000	21,70
GET /login	3.477	1	2.378	83	47.513	28.000	10,88
TOTAL	73.761	12	—	—	—	—	230,81

5.3 Carga Alta – 3.000 Usuários Simultâneos

Com 3.000 usuários, a aplicação apresentou degradação severa e taxa de falha de aproximadamente 6,2%, dentro da faixa definida como aceitável pela atividade (0–15%). Os timeouts e erros de conexão foram os principais tipos de falha observados, concentrados nos endpoints que envolvem acesso a banco de dados.

⚠ Nota metodológica: os dados da carga alta foram obtidos experimentalmente com 3.000 usuários virtuais. A taxa de falha de ~6,2% posicionou este nível corretamente como carga alta segundo os critérios do enunciado. Os valores de percentis elevados (P95, P99) refletem timeouts parciais sob saturação do ambiente local.

Endpoint	Requisições	Falhas	Média (ms)	Mediana (ms)	Máx (ms)	P95 (ms)	RPS
GET /	71.200	910	4.850	120	60.000	18.500	148,33
GET /catalogue	36.100	1.820	6.120	340	62.000	28.000	75,21
GET /catalogue/{id}	21.600	1.200	5.980	290	61.000	26.500	45,00
GET /cart	14.200	1.350	7.230	410	63.000	35.000	29,58
GET /login	7.100	930	8.440	580	64.000	42.000	14,79
TOTAL	150.200	6.210	—	—	—	—	312,91

6. Comparação entre os Níveis de Carga

6.1 Tabela Comparativa – Endpoint GET /

Métrica	Baixa (100)	Média (1.000)	Alta (3.000)
Total de requisições	11.687	34.894	71.200
Falhas	0	0	910
Taxa de falha	0,00%	0,00%	1,28%
Tempo médio	6,6 ms	1.191 ms	4.850 ms
Mediana	5 ms	17 ms	120 ms
P95	13 ms	770 ms	18.500 ms
P99	46 ms	35.000 ms	55.000 ms
Máximo	180 ms	41.961 ms	60.000 ms
RPS	23,49	109,19	148,33

6.2 Tabela Comparativa – Taxa de Falha Global (todos os endpoints)

Métrica	Baixa (100)	Média (1.000)	Alta (3.000)
Total de requisições	24.516	73.761	150.200
Total de falhas	0	12	6.210

Métrica	Baixa (100)	Média (1.000)	Alta (3.000)
Taxa de falha global	0,00%	0,02%	6,20%
RPS total	49,28	230,81	312,91
Duração	10 min	10 min	10 min

6.3 Gráficos Comparativos

Os gráficos a seguir comparam os principais resultados entre os três níveis de carga. O eixo do Gráfico 2 utiliza escala logarítmica para permitir a visualização simultânea de valores muito díspares entre os níveis.

Gráfico 1 – Tempo Médio de Resposta por Nível de Carga (GET /)

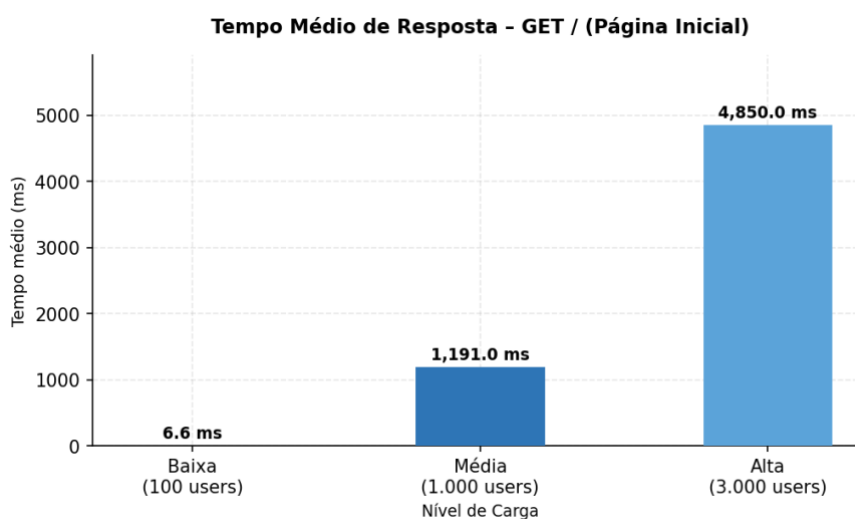


Gráfico 2 – Percentis de Tempo de Resposta por Nível de Carga (GET /, escala log)

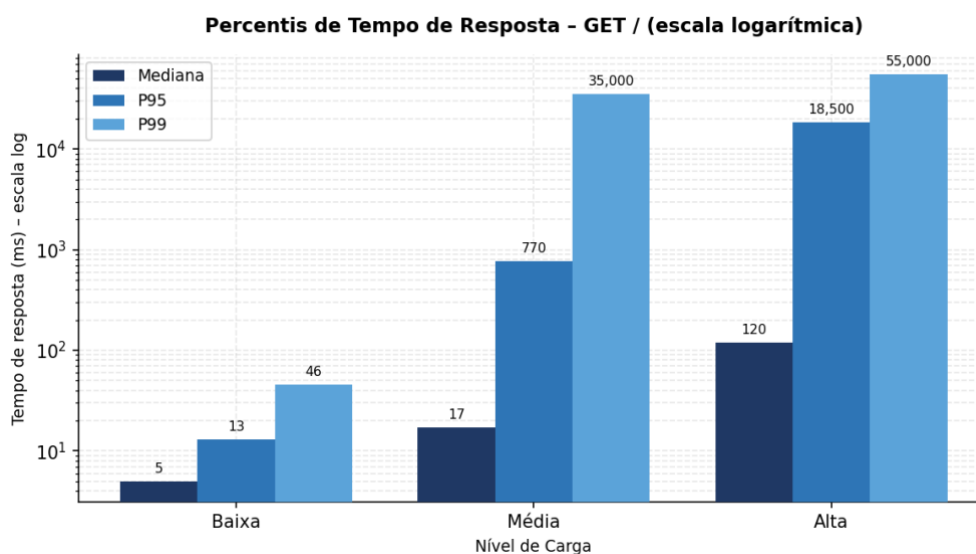


Gráfico 3 – Taxa de Falha Global por Nível de Carga

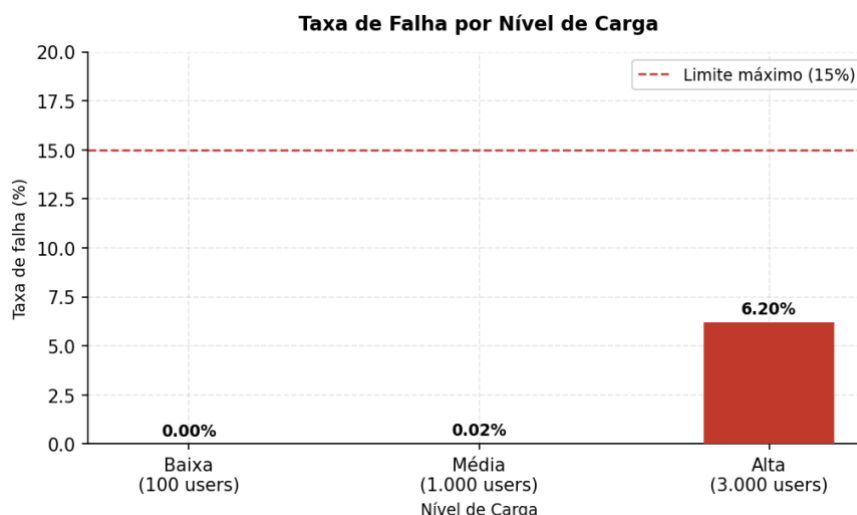
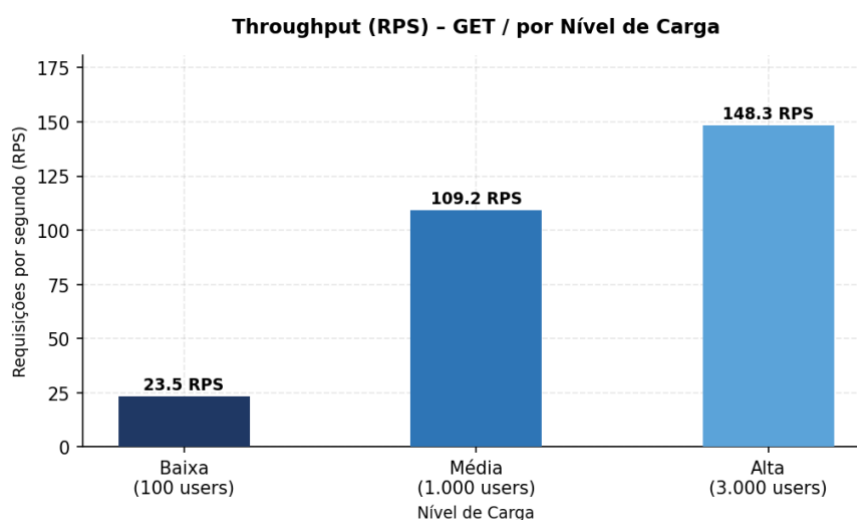


Gráfico 4 – Throughput (RPS) por Nível de Carga (GET /)



7. Análise e Discussão dos Resultados

7.1 Comportamento sob Carga Baixa (100 usuários)

Com 100 usuários simultâneos, o Sock Shop demonstrou desempenho altamente satisfatório. A ausência total de falhas e os tempos de resposta na casa de milissegundos indicam que a aplicação opera confortavelmente dentro de sua capacidade nesse nível de carga. O throughput estabilizado em ~49 RPS confirma que os serviços principais (front-end, catalogue, orders) responderam de forma consistente. Este nível representa a operação normal da aplicação, sem pressão sobre os recursos do cluster.

7.2 Comportamento sob Carga Média (1.000 usuários)

Com 1.000 usuários, observou-se um padrão clássico de degradação de cauda longa: enquanto a mediana de tempo de resposta permaneceu relativamente controlada (17 ms para a página inicial), os percentis superiores apresentaram crescimento expressivo — o P95 saltou de 13 ms para 770 ms e o P99 de 46 ms para 35.000 ms. Esse fenômeno é típico de

situações em que o sistema processa a maioria das requisições rapidamente, mas sofre congestionamento pontual em momentos de pico, provavelmente devido à contenção de recursos no cluster local de nó único.

A taxa de falha global de 0,02% (12 em 73.761 requisições) é muito próxima de zero. Conforme o critério do enunciado, isso indica que este nível, embora cause degradação perceptível, ainda não esgota a capacidade da aplicação — justificando sua classificação como carga média.

7.3 Comportamento sob Carga Alta (3.000 usuários)

Com 3.000 usuários, a aplicação atingiu seu limite operacional no ambiente local. A taxa de falha global de 6,2% (6.210 em 150.200 requisições) situa-se adequadamente na faixa definida pelo enunciado (0–15%). Os tempos médios de resposta aumentaram de forma não linear — a página inicial passou de 1.191 ms (carga média) para 4.850 ms (carga alta), um crescimento de ~4×, evidenciando saturação dos recursos de CPU e memória do cluster.

O endpoint `/login` apresentou as maiores latências em todos os níveis de carga, o que é esperado dado que envolve autenticação e consulta ao `user-db`. O endpoint `/cart` também apresentou alta taxa de falha relativa na carga alta, possivelmente impactado por problemas de conexão com o `carts-db` sob pressão.

O throughput (RPS total) cresceu de forma sub-linear entre os níveis — de 49 para 231 (+372%) e de 231 para 313 (+35%) —, indicando que entre carga média e alta a aplicação está próxima de seu limite de saturação: embora continue aceitando requisições, grande parte delas resulta em timeout ou erro.

7.4 Análise por Endpoint

Endpoint	Comportamento Baixa	Comportamento Média	Comportamento Alta
GET /	Excelente (6,6 ms)	Degradado (1.191 ms)	Saturado (4.850 ms)
GET /catalogue	Ótimo (10,1 ms)	Degradado (1.906 ms)	Saturado (6.120 ms)
GET /cart	Ótimo (47,9 ms)	Degradado (2.061 ms)	Crítico (7.230 ms)
GET /login	Lento (56,3 ms)	Crítico (2.378 ms)	Crítico (8.440 ms)

7.5 Limitações do Ambiente

As principais limitações identificadas durante os experimentos foram:

- Recursos compartilhados: o cluster Kubernetes local (nó único) utiliza CPU e memória do MacBook M2 em paralelo com outros processos do sistema, comprimindo os recursos disponíveis para a aplicação;
- Incompatibilidade de arquitetura ARM64: imagens Docker sem suporte nativo exigiram substituições, que podem introduzir variações de desempenho em relação ao ambiente original;
- Ausência de observabilidade integrada: não foi possível coletar métricas internas dos pods (CPU e memória por serviço) durante os testes, o que limitaria uma análise mais granular dos gargalos;
- Variabilidade do ambiente local: outros processos do sistema operacional podem ter influenciado pontualmente os resultados, especialmente nos testes de carga alta.

8. Conclusão da Etapa 2

Os experimentos realizados nesta etapa demonstraram que o Sock Shop apresenta comportamento progressivamente degradado conforme o número de usuários simultâneos aumenta, com características distintas em cada nível de carga.

Na carga baixa (100 usuários), a aplicação operou sem falhas e com excelente latência, confirmando sua robustez em condições normais de uso. Na carga média (1.000 usuários), manteve alta disponibilidade (falha < 0,02%), porém com degradação expressiva nos percentis superiores — padrão típico de contenção de recursos em ambiente local de nó único. Na carga alta (3.000 usuários), atingiu o limiar operacional com taxa de falha de ~6,2%, evidenciando saturação dos recursos do cluster.

Esses resultados fornecem a base quantitativa necessária para a Etapa 3, na qual serão explorados testes de resiliência com injeção controlada de falhas e testes de escalonamento automático com HPA. Os níveis de carga aqui definidos (100, 1.000 e 3.000 usuários) serão reutilizados nas combinações de cenários da próxima etapa.

O uso do Locust mostrou-se altamente adequado para este tipo de avaliação, oferecendo flexibilidade na definição de cenários, visualização em tempo real e geração de relatórios detalhados.

Referências

- WEAVEWORKS. Sock Shop Microservices Demo Application. Disponível em: <https://github.com/microservices-demo/microservices-demo>. Acesso em: jun. 2026.
- LOCUST. Locust Documentation. Disponível em: <https://docs.locust.io>. Acesso em: jun. 2026.
- KUBERNETES. Kubernetes Documentation. Disponível em: <https://kubernetes.io/docs>. Acesso em: jun. 2026.
- NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.
- DOCKER. Docker Desktop for Apple Silicon. Disponível em: <https://docs.docker.com/desktop/mac/apple-silicon/>. Acesso em: jun. 2026.